

CONTEXTGRAPH: REPAIR MEMORY FOR MISSING REQUIREMENTS

Anonymous authors

Paper under double-blind review

ABSTRACT

Past repairs record requirements that a new issue may leave unstated. We study a repair procedure that retrieves this experience before editing and uses it to complete the repair requirements. In complete-agent comparisons on three Django development tasks, repairs pass official verification with or without memory, but only the history-informed repairs satisfy additional historical requirements together with the current repair. The added behavior preserves query parameters, removes redundant uniqueness, or honors an explicit destination timezone. A fresh four-way comparison on two known benefit cases holds the historical source fixed and supplies no memory, a behavioral requirement, an implementation reference, or both. The requirement alone suffices for the observed timezone repair. On the constraint task, one requirement-only agent changes the same-column trigger to constraints on different column sets and misses the historical behavior; a later fixed attempt with the identical input preserves that trigger and the behavior. Further development cases expose incorrect applicability and excessive preservation under both requirements and edit advice. The useful memory must supply the missing behavior and enough information to apply it under the right conditions; a plausible requirement sentence alone does not ensure that these conditions survive use.

1 INTRODUCTION

The issue description tells a coding agent what is currently reported as broken. It need not enumerate every behavior a complete repair should preserve. A past repair can supply a requirement absent from the new issue, allowing memory to improve a repair that already passes the target’s tests.

Consider an administrative redirect that must preserve a deployment prefix. Changing its destination from `request.path_info` to `request.path` fixes that reported fault. However, appending a slash to the resulting path still discards query parameters. A different repair of the same API records this omitted behavior and replaces path concatenation with `request.get_full_path(force_append_slash=True)`. The useful information in this experience is the requirement to retain the query, which must hold together with the deployment-prefix requirement.

This view leads to a direct repair procedure: supply the current issue and a relevant historical repair before generating code. We compare this procedure with supplying the same experience after an initial repair, and with an additional review using only the current issue. The comparison determines whether the corrective information requires a separate revision stage.

2 USING HISTORY TO COMPLETE THE REPAIR REQUIREMENTS

Supply the requirement during generation. The repair writer receives the target issue and retrieved repair memory before editing. Memory is retrieved from the source partition using frozen BM25 rankings or cosine similarity with Qwen3-Embedding-8B (Zhang et al., 2025). The historical record describes an additional behavior for the new repair to preserve. We use either the stored distilled rule or its original source issue and patch as the memory input. We also test a generated behavioral requirement sentence without the source implementation. We study two writers: a method

Table 1: Django 14404 development comparison. Every row passes official verification: 2 fail-to-pass and 323 pass-to-pass tests. Calls include the initial generation when a revision is used.

Procedure	Calls	Prefix	Query	Joint
Issue only	1	Pass	Fail	Fail
Issue + ordinary review	2	Pass	Fail	Fail
History in revision	2	Pass	Pass	Pass
History in first call	1	Pass	Pass	Pass

generator supplied with the initial implementation, and a complete SWE-agent that browses the repository, edits code, runs tools, and submits a patch. The method generator returns one repaired method while preserving its public signature and decorator; the repository agent controls its own sequence of actions.

Measure the requirements together. We immediately evaluate every generated patch with the official SWE-bench verifier (Jimenez et al., 2024). To measure the behavior contributed by the source repair, a separate writer generates a short check from its patch and relevant repaired code. A normal execution changing from false on the buggy source to true on the repaired source demonstrates the historical behavior on the supplied inputs. We then execute this check unchanged on the patched target. An issue check measures the reported requirement, and a joint input exercises it together with the historical requirement.

3 EVIDENCE: A HISTORICAL REQUIREMENT COMPLETES THE REPAIR

We use the training side of a SWE-bench Verified split with 280 source tasks, 70 development tasks, and a separate 150-task evaluation partition unused by these experiments. The development comparison pairs source Django 16612 with target Django 14404. The target issue asks for support of `FORCE_SCRIPT_NAME` in administrative redirects; the source records query preservation in the same method.

History supplies information that ordinary review misses. DeepSeek V4 Pro receives the issue and initial implementation, including the available request-path APIs. Its first repair of `AdminSite.catch_all_view()` fixes the prefix but discards query strings. Two independent revisions start from this candidate: issue-based review retains it, while the historical issue and patch lead to `get_full_path(force_append_slash=True)`, preserving both requirements. These method-generation calls share the model, temperature 0.7, and a 16,384-token output allowance.

The information can be supplied during generation. Adding the identical history to the initial prompt, without a candidate or diagnostic feedback, produces the same method AST as historical revision (Table 1). It uses 4,482 provider-reported tokens versus 13,601 for the initial call and historical revision combined. On this case, supplying history initially reaches the same measured behavior without a separate revision stage.

Both requirements hold on a joint input. An independently prepared check combines the deployment prefix with repeated query parameters and percent-encoded values: `keep=a%2Fb&keep=c%20d`. The input is withheld from all writers and executed after their official verifications. The history-guided revision and the direct history generation preserve the exact prefix and query string; the initial and issue-reviewed candidates lose the query string.

A second application avoids a failed initial repair. The public issue in Pytest 7236 concerns a skipped test method whose `tearDown()` still runs under `--pdb`. Source 10081 records the corresponding class-level skip requirement. One call supplied with this history adapts the source helper to the target’s existing `__unittest_skip__` flags and checks both the method and its containing class. It passes official verification (1 fail-to-pass and 51 pass-to-pass tests), both skip checks, and a joint session that also checks a normal test’s lifecycle. The earlier ordinary initial

call fails; one issue-only diagnostic revision then repairs both skip conditions using the runtime skip state. Direct history reaches these measured behaviors in the first repair call, using 8,346 provider-reported tokens compared with 25,883 for that observed two-call sequence.

The gap also occurs in a complete repository agent. We run a fresh pair of SWE-agent attempts on Django 14404, supplying either no memory or the same first-ranked source issue and patch before the agent begins. These runs use DeepSeek V4 Pro0423 through OpenRouter, temperature 0, and a 32,768-token output allowance per call; the preceding method-generation comparisons use the 0813 snapshot. Both agents browse the repository, execute tests, and submit normally. Both pass all 2 fail-to-pass and 323 pass-to-pass tests, but only the history-informed repair preserves the query and passes the joint check. The ordinary agent’s actual observations include both the existing redirect helper and the implementation that retains `QUERY_STRING`, before it edits the method. With history, the agent preserves a behavior whose implementation was already visible to the ordinary agent.

The requirement survives distillation. The existing first-ranked Flat rule explicitly describes query loss and supplies `get_full_path(force_append_slash=True)`. We run a second fixed pair of complete-agent attempts: one receives the unchanged three-entry Flat packet, while the other receives the same packet with only its first entry replaced by the original source issue and patch. The remaining entries and the delivery point are identical. Both repairs pass official verification and all three behavior checks, and their submitted patches are byte-identical. In this development comparison, either representation supplies enough information to preserve the current and historical requirements together.

The requirement need not prescribe the implementation. The distilled redirect rule includes the repair API, so we further compare two fresh agents starting from the same completed ordinary Django 14404 patch. Both receive the source-generated sentence, “The redirect location from AdminSite.catch_all_view includes the original query string.” One receives this sentence alone; the other also receives the unchanged generated check and its failed observations on the candidate. The agents submit byte-identical patches after 27 and 28 calls, respectively, and both pass official verification and all three behavior checks. The sentence-only agent locates `get_full_path()` in the current repository and uses it to repair the redirect. In this case, memory supplies the desired behavior while the agent finds its implementation; providing the executable feedback adds no observed behavior.

A historical requirement can extend to a neighboring operation. We run a fixed cohort of six development tasks selected for high Qwen retrieval similarity. The issue-only and Qwen Flat agents both resolve all six tasks under official verification. The following two cases identify additional historical behaviors that distinguish their repairs despite this identical official score. In Django 12708, the issue reports ambiguous deletion of an ordinary index when a unique constraint uses the same columns. The memory packet contains three unchanged Flat rules, with source 15732 ranked first. That source concerns deletion of a unique constraint sharing its column with a primary key. Both agents fix ordinary-index selection and pass official verification. The memory-informed agent also excludes primary keys when deleting unique constraints. A PostgreSQL check generated from source evidence covers redundant uniqueness beside a primary key or a field-level unique constraint. Both inputs distinguish the source versions; the unchanged check fails on the ordinary target repair and passes on the memory-informed repair, with original constraints retained. A combined input creates an ordinary index, redundant uniqueness, and a primary key on the same column. On this PostgreSQL input, ordinary-index deletion already succeeds on the target base. Both repairs retain that behavior, but only the memory-informed repair also deletes the redundant uniqueness, retaining the primary key and original data. The useful historical information here supplies an additional operation’s requirement, beyond the operation singled out by the current failure.

History can complete a different argument of the same operation. Django 11138 asks date conversions to respect the database’s configured source timezone. Source 13363 instead requires `TruncDate` and `TruncTime` to honor an expression’s explicit destination timezone. Both complete-agent repairs pass official verification and correctly interpret a database configured as UTC+1. Only the Qwen Flat repair also honors the explicit destination. A check generated from

Table 2: One fresh complete-agent attempt per input and known development task. All eight pass official verification. The two joint columns measure the original current/historical combinations. The MySQL SQL-generation and named-zone ORM cases are additional conditions specified after inspecting completed timezone repairs and then executed on every timezone submission.

Memory input	Constraint joint	Timezone joint	MySQL cases	Named-zone cases
None	Fail	Fail	Pass	Fail
Requirement	Fail	Pass	Pass	Pass
Implementation	Pass	Pass	Pass	Fail
Both	Pass	Pass	Fail	Pass

source evidence distinguishes both source versions using a New York date aggregation and a Tokyo time aggregation; it runs unchanged on the target, failing on the ordinary repair and passing on the memory-informed repair. An independent SQLite input combines a UTC+1 database with an explicit New York destination. Only the memory-informed repair returns the correct date and time. The current issue and the retrieved history supply independent requirements for the two ends of the conversion.

Executing the historical requirement can guide a further repair. We start two fresh agents from the identical completed ordinary Django 11138 patch, with equal model, tools, and budget. One receives the issue and candidate for additional review; the other also receives the generated source check and its two failed observations, without the historical patch or the separate joint input. The ordinary review uses 47 calls and returns the original patch unchanged. The feedback-guided agent reproduces both failures, changes the two truncation methods to use `self.get_tzname()`, and submits after 43 calls. Both final patches pass official verification (1 fail-to-pass and 73 pass-to-pass tests), but only the feedback-guided revision passes both historical checks and the joint timezone input. Thus, in this intervention on a known development omission, an executed historical requirement supplies corrective information that further issue-based review does not recover.

Which information makes the requirement usable? We select the two preceding benefit cases and run four fresh complete agents per task, fixing the single historical source and its delivery before editing. Two source-only constructions separate a behavioral requirement from an implementation reference; the combined input concatenates them unchanged. The reference also conveys behavior, so the comparison concerns these actual inputs rather than disjoint information categories. All eight repairs pass official verification (Table 2). On Django12708, only Implementation and Both also remove redundant uniqueness while preserving the primary key. The Requirement agent tests a field with `unique=True`, but places `unique_together` on a different, two-column set. This nearby passing test loses the same-column condition stated in memory. On Django11138, all three memory inputs preserve the explicit destination timezone, while the fresh no-memory repair omits it. The Requirement agent finds `self.get_tzname()` in the target implementation without being given that helper in memory.

The timezone repairs also differ on further conditions. Both still generates an unnecessary MySQL conversion between identical zones; Implementation instead misinterprets a named Paris source timezone, producing New York time 19:21 instead of 17:30 on a summer input. Requirement passes both additional diagnostics. These concrete omissions distinguish carrying a requirement, applying its triggering condition, and implementing the resulting behavior correctly. A subsequent fixed comparison starts two new agents from the original constraint-task base: the unchanged Requirement input, or that input plus a concrete example previously validated on the two historical source versions. The new Requirement agent reproduces the same-column conflict, observes two matching constraints, and adds generated-name disambiguation. Both submissions pass official verification and the independent joint check; adding the example yields no observed behavior gain in this pair. Thus the earlier failure does not establish that an implementation reference is necessary: the same requirement can yield different repairs depending on how its trigger is reconstructed.

4 APPLICABILITY IS PART OF REPAIR MEMORY

We compare historical requirements with concrete edit advice on three further development tasks, holding the three retrieved sources per task fixed. Both forms resolve Django 16819 and Django 12858, and fail Xarray 7229: two of three official successes per arm. On Django 16819, both repairs incorrectly cancel `RemoveIndex` followed by `AddIndex` whenever the name is unchanged. A SQLite diagnostic, motivated by the first agent’s intermediate actions, exposes the lost effect: removing an index on `pink` and adding the same name on `weight` should change the indexed column. Both submitted repairs reduce this sequence to no operations, leaving `pink` indexed. The original version preserves the replacement; both repairs also preserve the add-then-remove and unchanged-definition comparison cases.

The matched comparison on Xarray 7229 illustrates excessive preservation. Both agents repair the common coordinate-attribute reproduction and pass all 280 official pass-to-pass tests, but fail the official target test. The edit-advice repair retains the condition array’s coordinate attributes when both value arguments to `where` are scalars; the requirements repair already does so when only the first is scalar. Neither memory form establishes the boundary between required preservation and required change in these repairs.

That boundary can already be wrong in the historical memory. We generate source-only contracts with applicability, intended-change and preservation fields. For Xarray 4094, the contract claims a stack/unstack roundtrip preserves sample coordinates when variables have only sample dimensions. The literal official source patch restores the reported three-element dataset, but a manually chosen one-element input loses the sample dimension and coordinate. The original source raises a merge error on both inputs. The singleton satisfies the generated condition, so this package is rejected before target use. Explicit contract fields alone do not establish the scope supported by the repaired source.

5 RELATED WORK

ExpeL distills reusable insights from experience, and Getafix learns applicable edit patterns from human repairs (Zhao et al., 2024; Bader et al., 2019). Memory Transfer Learning compares memory abstraction across coding domains and identifies procedural knowledge as a major source of transfer (Kim et al., 2026). ExpeRepair uses historical test demonstrations and reflective insights to generate reproduction and boundary tests (Mu et al., 2026). STAIR abstracts trajectories into hierarchical plans supplied before repair, including guidance that completes partial fixes (Xu et al., 2026). MIRA separates SQL corrections into reusable requirements and uses database checks to guide their activation and adaptation (Liu et al., 2026). TRACE compiles user corrections into atomic rules and runtime checks for later coding-agent tasks (Zhou et al., 2026). Working-set interventions study how supplying required facts supports code generation and how written conventions can override working code (Mohammadi et al., 2026). Jiang et al. (2026) study late requirement disclosure in coding-agent sessions and controlled tasks. Our development comparison examines when a retrieved historical requirement helps preserve current and historical behaviors together, including behavior absent from the official target tests.

AI USE STATEMENT

Generative AI assisted literature review, method development, implementation, experiments, analysis, and writing.

REFERENCES

- Johannes Bader, Andrew Scott, Michael Pradel, and Satish Chandra. Getafix: Learning to fix bugs automatically, 2019. URL <https://arxiv.org/abs/1902.06111>.
- Bowen Jiang, Haowei Cheng, Yuhong Fu, Anne Koziolok, Jialong Li, and Weixing Zhang. Requirements after the first edit: Mining late requirement emergence and rework in real-world coding-agent sessions, 2026. URL <https://arxiv.org/abs/2609.03028v1>. Version 1.

- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Kangsan Kim, Minki Kang, Taeil Kim, Yanlai Yang, Mengye Ren, and Sung Ju Hwang. Memory transfer learning: How memories are transferred across domains in coding agents, 2026. URL <https://arxiv.org/abs/2604.14004v1>. Version 1.
- Yining Liu, Chenyu Yang, Boyan Li, Rui Mao, and Yuyu Luo. MIRA: Evidence-verified repair memory for text-to-sql correction, 2026. URL <https://arxiv.org/abs/2608.06950v1>.
- Bardia Mohammadi, Lars Klein, Aman Chadha, Akhil Arora, and Laurent Bindschaedler. The working set of a coding agent: Coherence debt in repository-scale tasks, 2026. URL <https://arxiv.org/abs/2608.16630v1>. Version 1.
- Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. ExpeRepair: Dual-memory enhanced LLM-based repository-level program repair, 2026. URL <https://arxiv.org/abs/2506.10484v2>. Version 2; accepted at FSE 2026.
- Yisen Xu, Jiayuan Zhou, Ruiqi Pan, and Tse-Hsun Chen. Reusing past repairs through hierarchical trajectory abstraction for coding agents, 2026. URL <https://arxiv.org/abs/2607.29658v1>. Version 1.
- Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models, 2025. URL <https://arxiv.org/abs/2506.05176v3>.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, 2024. URL <https://arxiv.org/abs/2308.10144v3>.
- Yujun Zhou, Kehan Guo, Haomin Zhuang, Xiangqi Wang, Yue Huang, Zhenwen Liang, Pin-Yu Chen, Tian Gao, Nuno Moniz, Nitesh V. Chawla, and Xiangliang Zhang. Getting better at working with you: Compiling user corrections into runtime enforcement for coding agents, 2026. URL <https://arxiv.org/abs/2606.13174v1>.